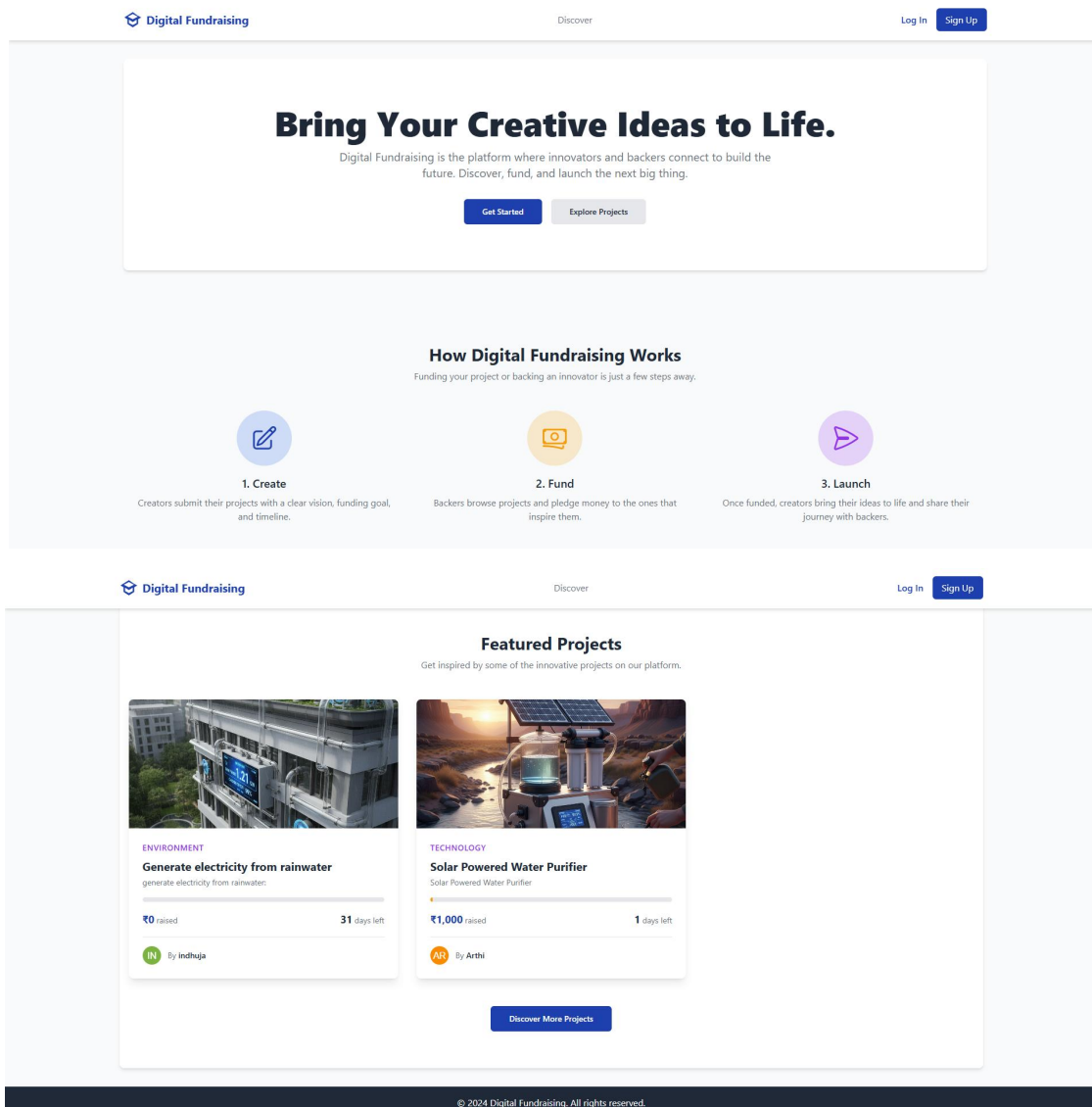


CHAPTER 6

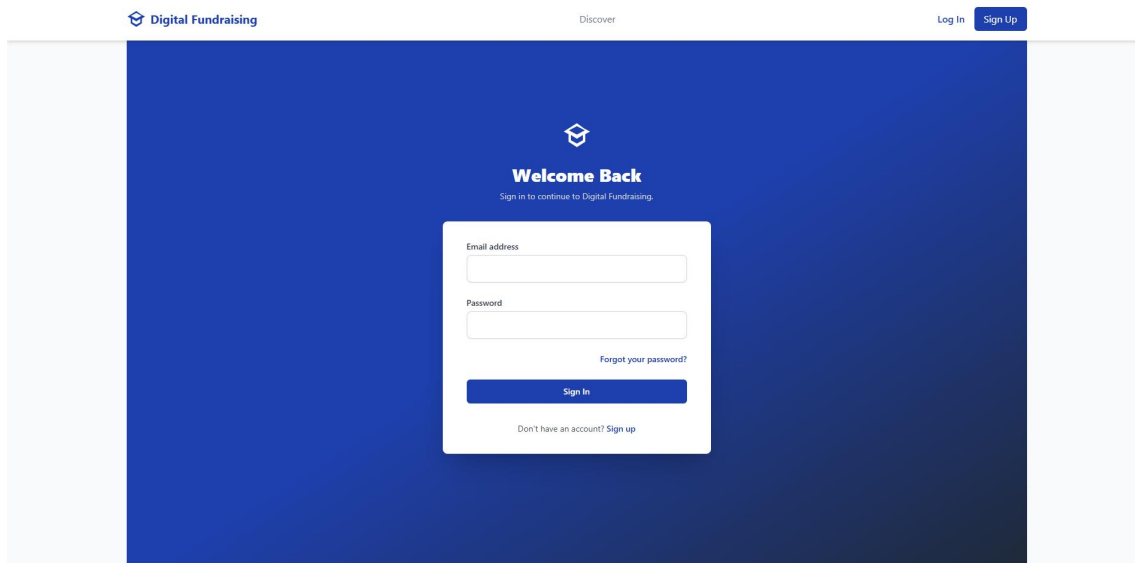
RESULT, DICUSSION AND CONCLUSION

6.1 TEST REPORTS:

6.1.1 LANDING PAGE:



6.1.2 LOGIN PAGE:



The screenshot shows the login page for Digital Fundraising. The header includes the logo, a 'Discover' link, and 'Log In' and 'Sign Up' buttons. The main content area has a blue gradient background with a white login form. The form includes fields for 'Email address' and 'Password', a 'Forgot your password?' link, a 'Sign In' button, and a link to 'Sign up' for new users.

Digital Fundraising Discover Log In Sign Up



Welcome Back
Sign in to continue to Digital Fundraising.

Email address

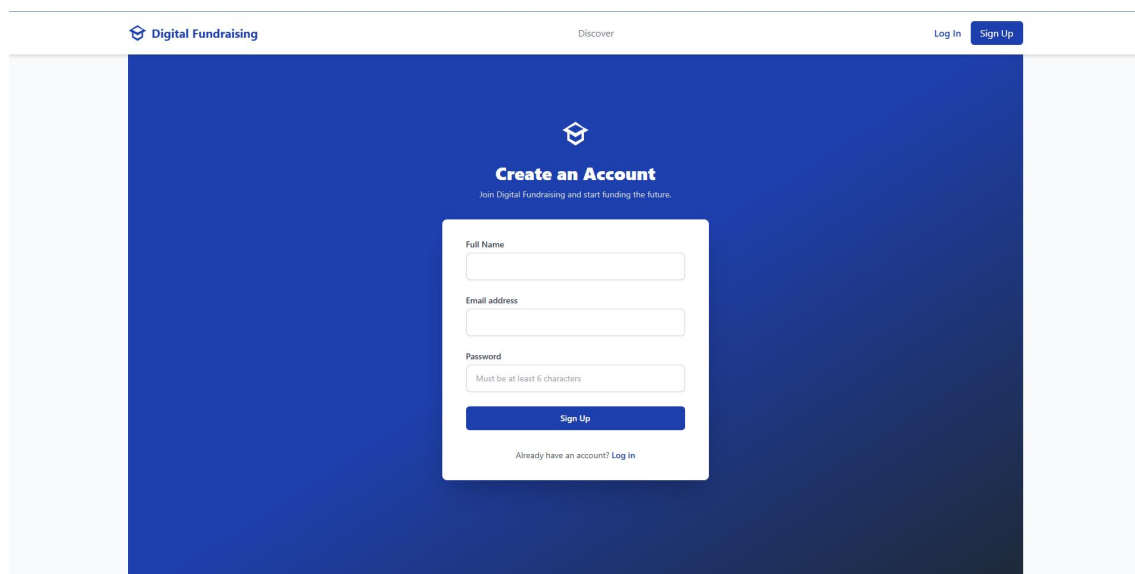
Password

[Forgot your password?](#)

Sign In

Don't have an account? [Sign up](#)

6.1.3 SIGNUP PAGE:



The screenshot shows the signup page for Digital Fundraising. The header is identical to the login page. The main content area has a blue gradient background with a white signup form. The form includes fields for 'Full Name', 'Email address', and 'Password' (with a note that it must be at least 6 characters), a 'Sign Up' button, and a link to 'Log in' for existing users.

Digital Fundraising Discover Log In Sign Up



Create an Account
Join Digital Fundraising and start funding the future.

Full Name

Email address

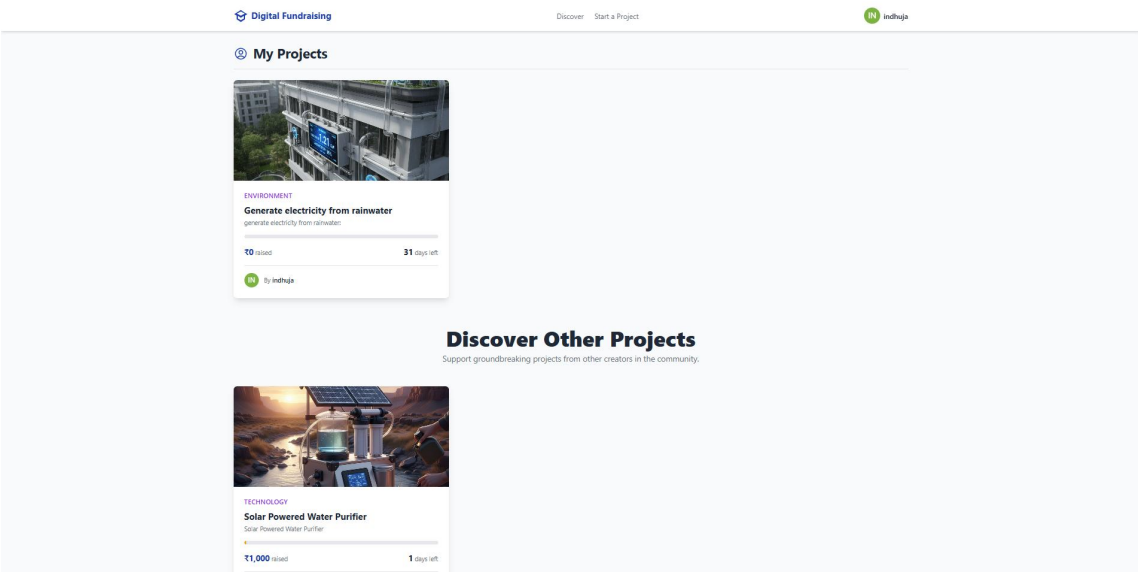
Password

Must be at least 6 characters

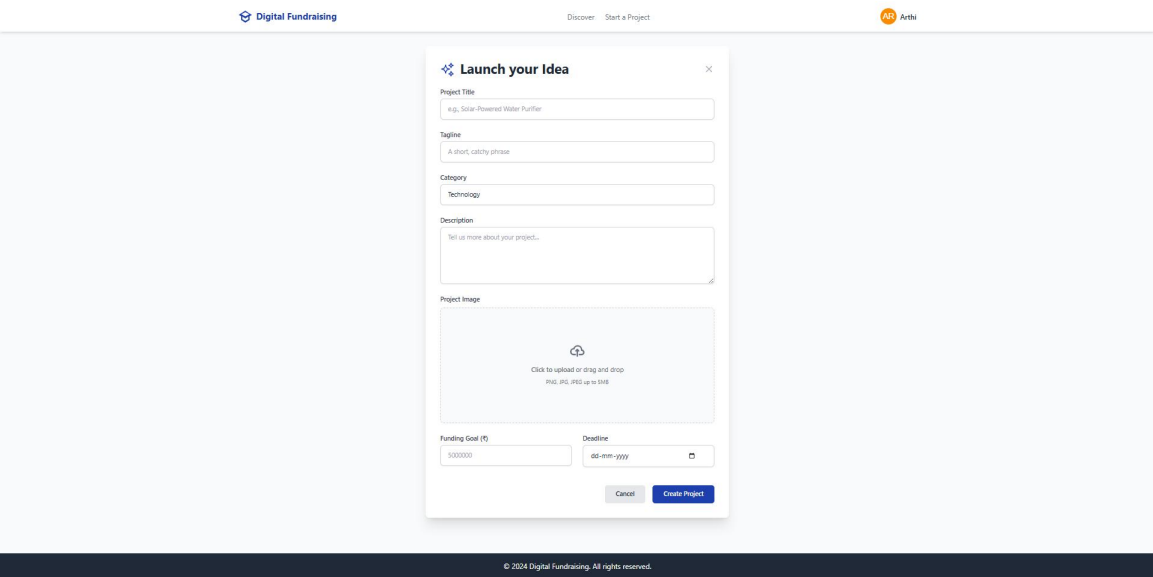
Sign Up

Already have an account? [Log in](#)

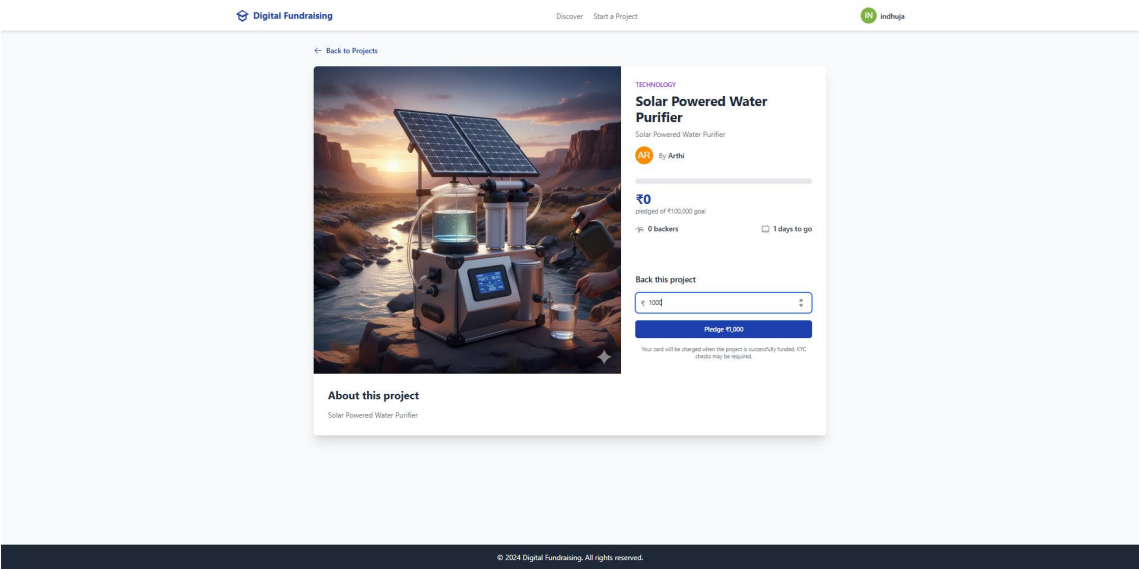
6.1.4 USER DASHBOARD:



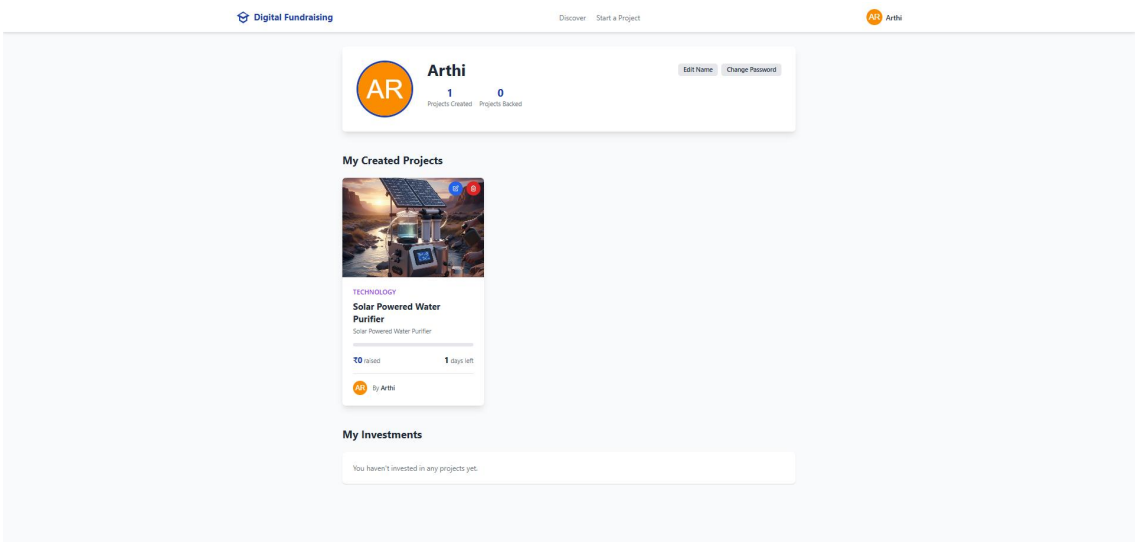
6.1.5 PROJECT CREATING SECTION:



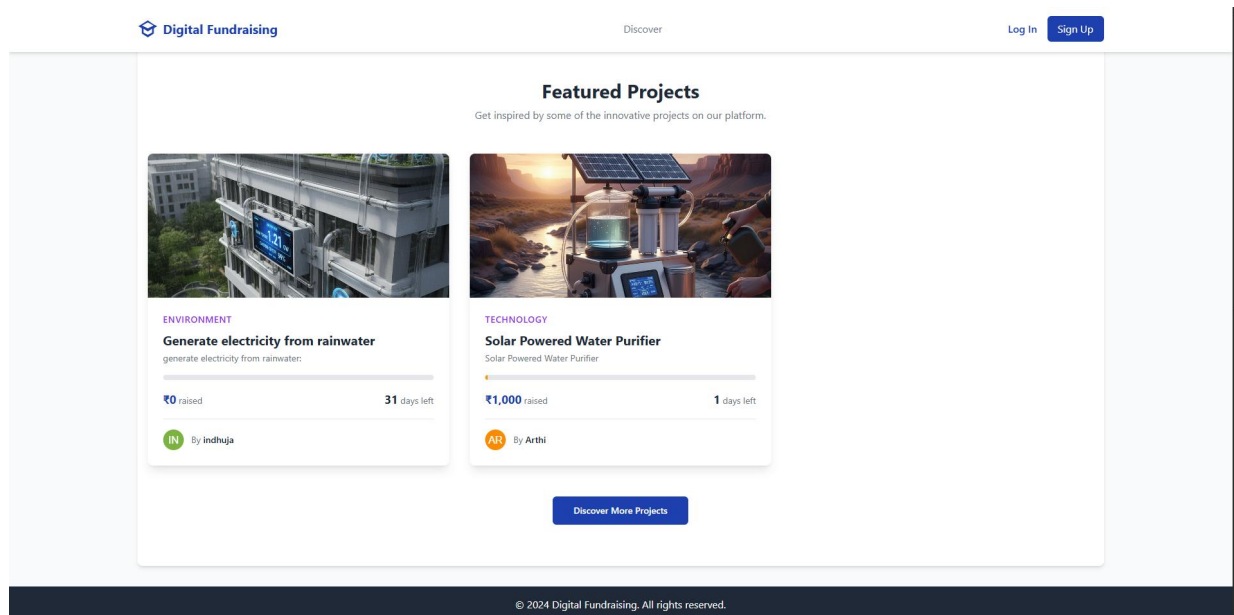
6.1.6 PROJECT PLEDGING SECTION:



6.1.7 USER PROFILE PAGE:



6.1.8 PROJECTS LIST PAGE:



6.2 USER DOCUMENTATION:

MAIN PAGE:

```
import React, { useState, useEffect } from "react";
import type { Project, Profile } from "./types";
import { View } from "./types";
import { AuthProvider, useAuth } from "./context/AuthContext";
import Navbar from "./components/Navbar";
import ProjectList from "./components/ProjectList";
import ProjectDetails from "./components/ProjectDetails";
import CreateProjectForm from "./components/CreateProjectForm";
import UserProfile from "./components/UserProfile";
import Login from "./components/Login";
import SignUp from "./components/SignUp";
import LandingPage from "./components/LandingPage";
import UserManagement from "./components/UserManagement";
import ForgotPassword from "./components/ForgotPassword";
import ResetPassword from "./components/ResetPassword";
import { supabase } from "./lib/supabaseClient";
```

```

const AppContent: React.FC = () => {
  const [currentView, setCurrentView] = useState<View>(View.Landing);
  const [selectedProjectId, setSelectedProjectId] = useState<string | null>(
    null
  );
  const [projects, setProjects] = useState<Project[]>([]);
  const [loadingProjects, setLoadingProjects] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [editingProject, setEditingProject] = useState<Project | null>(null);
  const { user, profile, isAdmin, loading: authLoading } = useAuth();

  const navigateTo = (view: View) => {
    window.scrollTo(0, 0);
    setCurrentView(view);
  };

  const fetchProjects = async () => {
    setLoadingProjects(true);
    setError(null);
    try {
      const { data, error } = await supabase
        .from("projects")
        .select(
          `
            *,
            profiles ( id, name, avatar_url ),
            investments ( user_id )
          `
        )
        .order("created_at", { ascending: false });

      if (error) throw error;
      setProjects(data || []);
    } catch (err: any) {

```

```

    console.error("Error fetching projects:", err);
    setError("Failed to load projects. Please try again later.");
  } finally {
    setLoadingProjects(false);
  }
};

```

```

useEffect(() => {
  fetchProjects();
}, []);

```

```

useEffect(() => {
  // This effect handles Supabase's password recovery flow.
  // When a user clicks the link in their email, they are redirected here
  // with a URL hash containing `type=recovery`.
  if (window.location.hash.includes("type=recovery")) {
    navigateTo(View.ResetPassword);
  }
}, []);

```

```

useEffect(() => {
  if (user) {
    if (
      currentUser === View.Landing ||
      currentUser === View.Login ||
      currentUser === View.SignUp ||
      currentUser === View.ForgotPassword ||
      currentUser === View.ResetPassword
    ) {
      navigateTo(View.Home);
    }
  } else {
    // Do not redirect if auth is still loading
    if (

```

```

!authLoading &&
(currentView === View.CreateProject ||
  currentView === View.Profile ||
  currentView === View.UserManagement)
) {
  navigateTo(View.Login);
}
}
}, [user, currentView, authLoading]);

```

```

const handleSelectProject = (projectId: string) => {
  setSelectedProjectId(projectId);
  navigateTo(View.ProjectDetails);
};

```

```

const handleCreateProject = async (
  newProjectData: Omit<
    Project,
    | "id"
    | "creator_id"
    | "current_funding"
    | "investors"
    | "profiles"
    | "investments"
    | "created_at"
    | "updated_at"
  >
) => {
  if (!user) {
    alert("You must be logged in to create a project.");
    return;
  }
  try {
    const { data, error } = await supabase

```

```

    .from("projects")
    .insert([ { ...newProjectData, creator_id: user.id } ])
    .select()
    .single();

if (error) throw error;

// Refetch projects to show the new one
await fetchProjects();
navigateTo(View.Home);
} catch (err) {
  console.error("Error creating project:", err);
  alert("Failed to create project.");
}
};

const handleInvest = async (projectId: string, amount: number) => {
  if (!user) {
    alert("You must be logged in to invest.");
    return;
  }
  try {
    // The previous client-side update was likely failing due to database permissions (Row-
    Level Security).
    // The correct and more secure approach is to use a Supabase RPC (database function)
    // that handles the logic on the backend atomically.
    const { error } = await supabase.rpc("handle_investment", {
      project_id_arg: projectId,
      amount_arg: amount,
    });

    if (error) throw error;

    // The database is now updated correctly via the RPC.

```

```

// Update the local state for immediate UI feedback.
setProjects((prevProjects) =>
  prevProjects.map((p) => {
    if (p.id === projectId) {
      // The 'investments' array from our fetch query only contains user_id.
      // We mimic that structure here for UI consistency.
      const newInvestments = [...p.investments, { user_id: user.id }];
      return {
        ...p,
        current_funding: p.current_funding + amount,
        investments: newInvestments,
      };
    }
    return p;
  })
);
} catch (err: any) {
  console.error("Error investing in project:", err);
  alert(
    `Investment failed: ${err.message}. Please ensure the 'handle_investment' function exists
in your Supabase project.`
  );
}
};

const handleEditProject = (project: Project) => {
  setEditingProject(project);
  navigateTo(View.CreateProject);
};

const handleDeleteProject = async (projectId: string) => {
  if (!user) {
    alert("You must be logged in to delete a project.");
    return;
  }

```

```

    }

    if (
      !confirm(
        "Are you sure you want to delete this project? This action cannot be undone."
      )
    ) {
      return;
    }

    try {
      const { error } = await supabase
        .from("projects")
        .delete()
        .eq("id", projectId)
        .eq("creator_id", user.id); // Ensure only the creator can delete

      if (error) throw error;

      // Remove project from local state
      setProjects((prev) => prev.filter((p) => p.id !== projectId));

      // If we're viewing the deleted project, go back to home
      if (selectedProjectId === projectId) {
        navigateTo(View.Home);
      }
    } catch (err: any) {
      console.error("Error deleting project:", err);
      alert(`Failed to delete project: ${err.message}`);
    }
  };

  const handleUpdateProject = async (updatedProjectData: any) => {
    if (!user || !editingProject) {

```

```

    alert("You must be logged in to update a project.");
    return;
}

try {
    const { data, error } = await supabase
        .from("projects")
        .update({
            title: updatedProjectData.title,
            tagline: updatedProjectData.tagline,
            description: updatedProjectData.description,
            category: updatedProjectData.category,
            image_url: updatedProjectData.image_url,
            funding_goal: updatedProjectData.funding_goal,
            deadline: updatedProjectData.deadline,
        })
        .eq("id", editingProject.id)
        .eq("creator_id", user.id)
        .select()
        .single();

    if (error) throw error;

    // Update project in local state
    setProjects((prev) =>
        prev.map((p) => (p.id === editingProject.id ? { ...p, ...data } : p))
    );

    setEditingProject(null);
    navigateTo(View.Home);
} catch (err: any) {
    console.error("Error updating project:", err);
    alert(`Failed to update project: ${err.message}`);
}

```

```
};
```

```
const handleProjectFormSubmit = (projectData: any) => {  
  if (editingProject) {  
    handleUpdateProject(projectData);  
  } else {  
    handleCreateProject(projectData);  
  }  
};
```

```
const handleCancelEdit = () => {  
  setEditingProject(null);  
  navigateTo(View.Home);  
};
```

```
const renderContent = () => {  
  if (authLoading) {  
    return (  
      <div className="flex justify-center items-center py-40">  
        <div className="text-center">  
          <svg  
            className="animate-spin h-10 w-10 text-primary mx-auto"  
            xmlns="http://www.w3.org/2000/svg"  
            fill="none"  
            viewBox="0 0 24 24"  
          >  
            <circle  
              className="opacity-25"  
              cx="12"  
              cy="12"  
              r="10"  
              stroke="currentColor"  
              strokeWidth="4"  
            ></circle>  
            <path
```

```

        className="opacity-75"
        fill="currentColor"
        d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12h4zm2 5.291A7.962 7.962 0 014
12H0c0 3.042 1.135 5.824 3 7.938l3-2.647z"
    ></path>
</svg>
<p className="mt-4 text-lg text-neutral">Authenticating...</p>
</div>
</div>
);
}

```

```

if (loadingProjects && projects.length === 0) {
  return <div className="text-center py-20">Loading projects...</div>;
}
if (error) {
  return <div className="text-center py-20 text-red-500">{error}</div>;
}

```

```

switch (currentView) {
  case View.Landing:
    const featuredProjects = projects.slice(0, 3);
    return (
      <LandingPage
        navigateTo={navigateTo}
        featuredProjects={featuredProjects}
        onSelectProject={handleSelectProject}
      />
    );
  case View.ProjectDetails:
    const project = projects.find((p) => p.id === selectedProjectId);
    return project ? (
      <ProjectDetails
        project={project}

```

```

      user={user}
      onInvest={handleInvest}
      onBack={() => navigateTo(View.Home)}
      onLoginRequest={() => navigateTo(View.Login)}
    />
  ) : (
    <ProjectList
      projects={projects}
      onSelectProject={handleSelectProject}
      user={user}
    />
  );
case View.CreateProject:
  if (!user) return null;
  return (
    <CreateProjectForm
      onSubmit={handleProjectFormSubmit}
      onCancel={handleCancelEdit}
      existingProject={editingProject}
      isEditing={!editingProject}
    />
  );
case View.Profile:
  if (!profile) return null;
  const userProjects = projects.filter(
    (p) => p.creator_id === profile.id
  );
  const userInvestments = projects.filter((p) =>
    p.investments.some((inv) => inv.user_id === profile.id)
  );
  return (
    <UserProfile
      profile={profile}
      createdProjects={userProjects}

```

```

        investedProjects={userInvestments}
        onSelectProject={handleSelectProject}
        onEditProject={handleEditProject}
        onDeleteProject={handleDeleteProject}
    />
);
case View.UserManagement:
    if (!isAdmin) {
        return (
            <div className="text-center py-20">
                <h2 className="text-2xl font-bold text-red-600">Access Denied</h2>
                <p className="mt-2 text-neutral">
                    You must be an administrator to view this page.
                </p>
            </div>
        );
    }
    return <UserManagement />;
case View.Login:
    return (
        <Login
            navigateToSignUp={() => navigateTo(View.SignUp)}
            navigateToForgotPassword={() => navigateTo(View.ForgotPassword)}
        />
    );
case View.SignUp:
    return <SignUp navigateToLogin={() => navigateTo(View.Login)} />;
case View.ForgotPassword:
    return (
        <ForgotPassword navigateToLogin={() => navigateTo(View.Login)} />
    );
case View.ResetPassword:
    return <ResetPassword navigateToLogin={() => navigateTo(View.Login)} />;
case View.Home:

```

default:

```
return (  
  <ProjectList  
    projects={projects}  
    onSelectProject={handleSelectProject}  
    user={user}  
  />  
);  
}  
};
```

```
return (  
  <div className="min-h-screen bg-gray-50 font-sans text-neutral-dark flex flex-col">  
    <Navbar navigateTo={navigateTo} />  
    <main className="container mx-auto px-4 py-8 flex-grow">  
      {renderContent()}  
    </main>  
    <footer className="bg-neutral-dark text-white text-center p-4">  
      <p>&copy; 2024 Digital Fundraising. All rights reserved.</p>  
    </footer>  
  </div>  
);  
};
```

```
const App: React.FC = () => {  
  return (  
    <AuthProvider>  
      <AppContent />  
    </AuthProvider>  
  );  
};
```

```
export default App;
```

AUTHENTICATION:

```
import React, {
  createContext,
  useState,
  useContext,
  ReactNode,
  useEffect,
} from "react";
import type { Profile } from "../types";
import { supabase } from "../lib/supabaseClient";
import type { User, Session } from "@supabase/supabase-js";

interface AuthContextType {
  user: User | null;
  profile: Profile | null;
  session: Session | null;
  loading: boolean;
  isAdmin: boolean;
  logout: () => Promise<void>;
  updateProfile: (updates: {
    name?: string;
    avatar_url?: string;
  }) => Promise<void>;
  changePassword: (newPassword: string) => Promise<void>;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export const AuthProvider: React.FC<{ children: ReactNode }> = ({
  children,
}) => {
  const [user, setUser] = useState<User | null>(null);
  const [profile, setProfile] = useState<Profile | null>(null);
```

```

const [session, setSession] = useState<Session | null>(null);
const [loading, setLoading] = useState(true);
const [isAdmin, setIsAdmin] = useState(false);

useEffect(() => {
  // Supabase's onAuthStateChange listener is the canonical way to manage session state.
  // It fires an initial event with the current session ('INITIAL_SESSION'),
  // and then for any subsequent changes (e.g., 'SIGNED_IN', 'SIGNED_OUT').
  const { data: authListener } = supabase.auth.onAuthStateChange(
    (_event, session) => {
      setSession(session);
      const currentUser = session?.user ?? null;
      setUser(currentUser);
    }
  );

  return () => {
    authListener?.subscription.unsubscribe();
  };
}, []);

// A separate effect to fetch the profile data whenever the user state changes.
// This has been refactored to be more robust against race conditions
// in async operations and unmounted components.
useEffect(() => {
  if (user) {
    setLoading(true);
    let isMounted = true;

    const fetchProfile = async () => {
      try {
        const { data, error, status } = await supabase
          .from("profiles")
          .select("*")

```

```

    .eq("id", user.id)
    .single();

    // Supabase postgres error convention: status 406 means no rows found,
    // which is not a server error for this use case.
    if (error && status !== 406) {
        throw error;
    }

    if (isMounted) {
        setProfile(data);
        // FIX: Make role check case-insensitive to handle variations like 'Admin'.
        setIsAdmin(data?.role?.toLowerCase() === "admin");
    }
} catch (err) {
    console.error("Error fetching profile:", err);
    if (isMounted) {
        setProfile(null);
        setIsAdmin(false);
    }
} finally {
    if (isMounted) {
        setLoading(false);
    }
}
};

fetchProfile();

return () => {
    isMounted = false;
};
} else {
    // If there's no user, clear the profile and stop loading.

```

```
    setProfile(null);
    setIsAdmin(false);
    setLoading(false);
  }
}, [user]);
```

```
const logout = async () => {
  const { error } = await supabase.auth.signOut();
  if (error) {
    console.error("Error logging out:", error);
  }
  // State will be cleared automatically by the onAuthStateChange listener.
};
```

```
const updateProfile = async (updates: {
  name?: string;
  avatar_url?: string;
}) => {
  if (!user) {
    throw new Error("User must be logged in to update profile");
  }
}
```

```
const { error } = await supabase
  .from("profiles")
  .update(updates)
  .eq("id", user.id);
```

```
if (error) {
  throw error;
}
```

```
// Update local profile state
setProfile((prev) => (prev ? { ...prev, ...updates } : null));
};
```

```
const changePassword = async (newPassword: string) => {  
  if (!user) {  
    throw new Error("User must be logged in to change password");  
  }  
}
```

```
const { error } = await supabase.auth.updateUser({  
  password: newPassword,  
});
```

```
if (error) {  
  throw error;  
}  
};
```

```
const value = {  
  user,  
  profile,  
  session,  
  loading,  
  isAdmin,  
  logout,  
  updateProfile,  
  changePassword,  
};  
return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;  
};
```

```
export const useAuth = (): AuthContextType => {  
  const context = useContext(AuthContext);  
  if (context === undefined) {  
    throw new Error("useAuth must be used within an AuthProvider");  
  }  
  return context;  
};
```

6.3 CONCLUSION:

The Digital Fundraising Project has been successfully developed and tested to provide a reliable, transparent, and efficient fundraising platform. The system bridges the gap between donors and organizations by offering an online space where causes can be showcased, donations can be made securely, and campaign progress can be tracked in real time.

The project achieves its goals by implementing features such as donor and organization management, campaign creation and monitoring, secure payment processing, and reporting modules. It reduces manual efforts, enhances transparency in fund allocation, and improves accessibility for both local and global donors.

This project not only demonstrates the importance of technology in social good but also highlights how digital solutions can overcome geographical barriers in fundraising. Overall, the Digital Fundraising System ensures that financial contributions are more structured, traceable, and impactful, thereby enhancing trust among stakeholders.

6.4 LIMITATIONS OF THE SYSTEM

- **Scalability Challenges** – The system is currently best suited for small to medium-scale organizations. Handling very large datasets, high traffic, or millions of users may require infrastructure upgrades.
- **Limited Payment Integration** – Only a few payment methods (such as card-based transactions) are supported. This reduces flexibility for international donors who prefer alternative payment gateways.
- **No AI-based Fraud Detection** – The system relies on basic validation and does not yet integrate machine learning models for fraud prevention, spam detection, or donor verification.
- **Dependence on Internet** – The platform cannot function without stable internet connectivity, restricting its usability in rural or remote areas.
- **No Mobile App Version** – Currently, the system exists as a web-based platform only, which may limit accessibility for mobile-first users.
- **Basic User Experience** – While functional, the system interface can be further improved with modern UX/UI design principles to enhance donor engagement.

- **Reporting Constraints** – The reporting system provides basic statistics but lacks advanced visual analytics and predictive insights.

6.5 FUTURE SCOPE OF THE PROJECT

- **Mobile Application Development** – Build native apps for Android and iOS to expand accessibility and convenience for on-the-go donors.
- **AI and Machine Learning Integration** – Implement fraud detection, donor behavior prediction, and recommendation systems for personalized fundraising campaigns.
- **Blockchain for Transparency** – Introduce blockchain technology to maintain tamper-proof transaction records, ensuring maximum transparency and donor trust.
- **Multiple Payment Gateways** – Integrate UPI, PayPal, Stripe, cryptocurrency, and other international gateways for wider donor participation.
- **Gamification Features** – Add reward systems, badges, or leaderboards for donors to encourage repeated contributions.
- **Advanced Reporting and Analytics** – Use data visualization dashboards and predictive analytics to give organizations deeper insights into campaign success and donor behavior.
- **Integration with Social Media Platforms** – Enable campaigns to be directly shared, promoted, and tracked via Facebook, Instagram, LinkedIn, and Twitter.
- **Automated Alerts and Notifications** – Use email, SMS, and WhatsApp notifications for donation confirmations, campaign updates, and fundraising milestones.
- **Multilingual and Multi-Currency Support** – Cater to international donors by supporting multiple languages and currencies.
- **CSR (Corporate Social Responsibility) Module** – Allow companies to participate and track contributions under their CSR initiatives.